KAVACH — On-device scam shield for Indian phone users

Bharat AI-SoC Challenge 2026–27 · Problem Statement 3 (Cybersecurity & Digital Safety) · Phase 1 technical report

November 2026

1. Summary
2. Problem and scope
3. Architecture
   - 3.1 Pipeline A — speech (Guardian, Arm CPU; NPU in Phase 2)
   - 3.2 Pipeline B — reasoning (Llama 3.2 1B, ExecuTorch, Arm CPU)
   - 3.3 The always-on fast path (both phones)
   - 3.4 User's phone modules
4. Arm-specific engineering
5. Data
6. Engineering quality
7. Results
   - 7.1 Text classifiers (held-out test splits)
   - 7.2 Call Guardian (call level, replayed in call time)
   - 7.3 End-to-end audio (host)
   - 7.4 Llama pipeline verification
   - 7.5 Still to measure on phones (Phase 2 week 1–2, scripts ready)
8. Limitations and threat model (summary)
9. Phase 2 plan
10. Reproducing everything

1. Summary

KAVACH listens to a suspicious call on speaker-phone (with consent), understands it on the phone, and stops the victim before money moves. It joins what it hears with the SMS, link, app and screen-share that follow, because real Indian scams move across channels. **Nothing leaves the device:** neither app requests the Android `INTERNET` permission.

What exists at the Phase 1 freeze (all in this repository):

- **Two Android apps** (Kotlin, no AndroidX, compile-checked against the Android 34 API, ExecuTorch 1.5 and sherpa-onnx 1.13.8 APIs): *KAVACH Guardian* (spare phone: streaming Hindi/Hinglish or English ASR, Call Guardian, Llama 3.2 1B advisor, spoken warnings) and *KAVACH Shield* (user's phone: Message Shield, Link Check, cross-channel kill-chain, full-screen STOP card, and Phase-2 previews of App Inspector and Screen-share Guard).
- **A Python reference implementation** of every on-device algorithm, with the Kotlin port proven identical by **196 parity checks**, including tick-by-tick replays of whole calls.
- **The Llama pipeline** (LoRA data, fine-tuning, Hugging Face → Meta conversion, ExecuTorch 4-bit export ladder, evaluation), **verified end to end** on a tiny Llama-3.2-shaped model: conversion is exact (max logit difference 3.6e-7), and the exported `.pte` reproduces Hugging Face token by token through the KV cache.
- **Evaluation harnesses** and first results, all on held-out data.

Headline results. These cover the always-on path (rules + classifier) *without* Llama; no false STOP was raised on any genuine call in any set:

Evaluation set	Scam calls reaching STOP	STOP before the money/OTP request	Genuine calls wrongly STOPped
Synthetic test, 1,000 calls, phrasings never trained on	**86.7 %**	77.1 %	**0 %**
Hand-written calls, 16 scam / 14 genuine	**93.8 %**	62.5 %	**0 %**
Locked held-out calls, 10 / 10, never used for tuning	**70 %**	50 %	**0 %**
End-to-end audio: TTS call → speaker-phone channel → ASR, 6 scam / 6 genuine runs	4 of 6 STOP, 6 of 6 ≥ CAUTION (a second audio realisation: 3 STOP, 5 ≥ CAUTION)	—	**0 of 6** (not even a CAUTION)

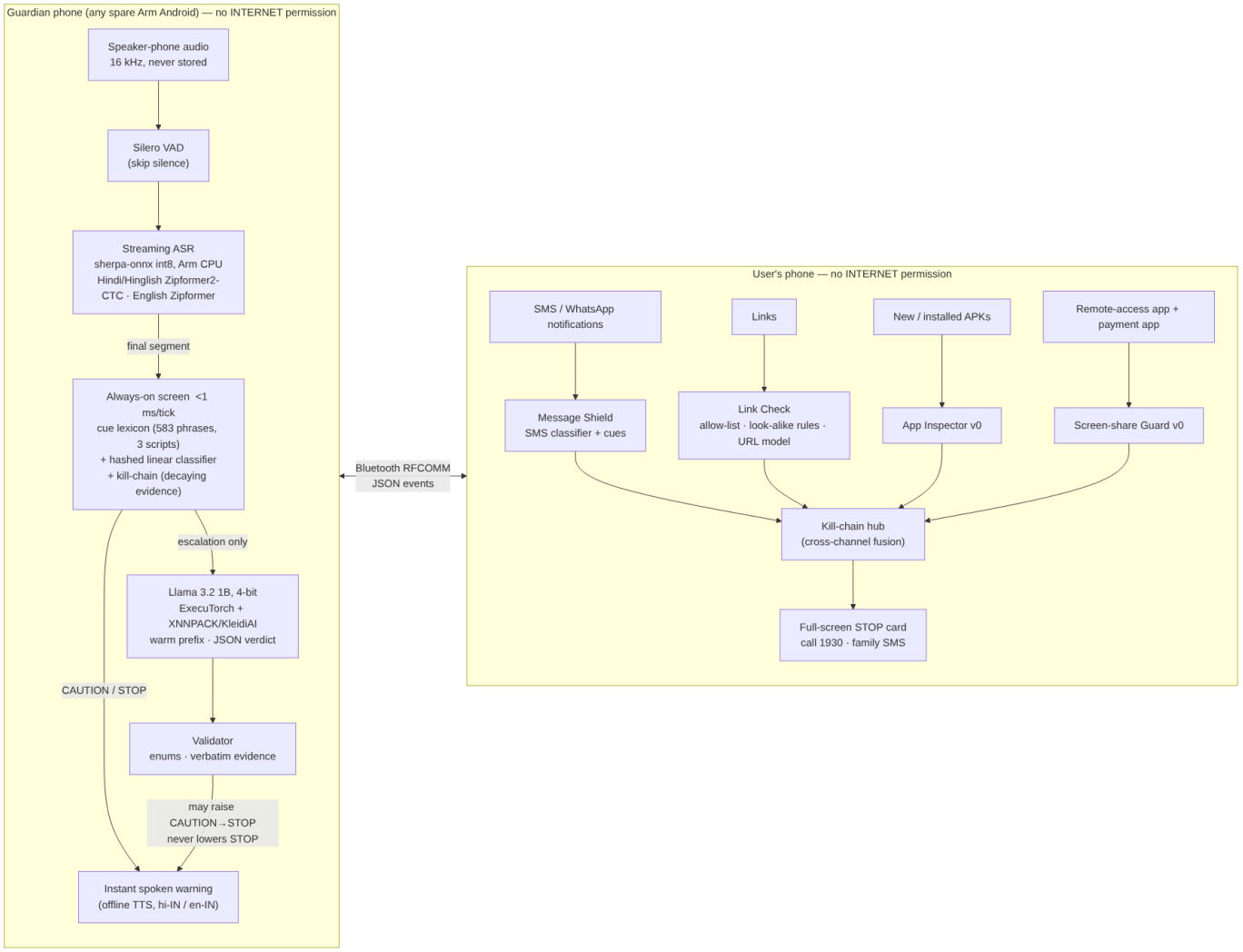
The honest reading: the fast path is safe, with no false STOPs, but on unseen phrasing it leaves 20–30 % of scam calls at CAUTION. That gap is exactly what the Llama 3.2 1B layer is for: it judges every CAUTION and may escalate it to STOP. Measuring that on phones with the fine-tuned model is the first Phase 2 milestone (§9). The other bottleneck is ASR quality on speaker-phone audio (§7.5).

2. Problem and scope

Indians lost ₹22,495 crore to cyber fraud in 2025 (MHA/I4C). “Digital arrest” calls alone caused 1.24 lakh cases and ₹1,918 crore of losses in 2024. A scam is a *sequence*: an authority claim, a threat or lure, isolation (“don’t disconnect, don’t tell your family”), an action (OTP, link, APK, screen share) and a payment. Single-channel filters see one fragment. On-device call-scam detection exists only on premium phones. KAVACH targets the budget and mid-range Android phones most families own, explains every alert in Hindi or English, and gives the next step (hang up, call 1930).

Since Android 10, third-party apps cannot capture call audio. The **Guardian** is therefore a spare Android phone placed beside the speaker-phone call; it works on any phone today. The production path is OEM or pre-loaded dialer integration.

3. Architecture



System architecture: two phones, two inference pipelines, no network

3.1 Pipeline A — speech (Guardian, Arm CPU; NPU in Phase 2)

Stage	Implementation	Notes
Capture	`AudioRecord`, `VOICE_RECOGNITION` source, 16 kHz mono, 100 ms chunks	AGC/noise suppression off; audio never written to storage
VAD	Silero VAD (sherpa-onnx), 512-sample windows	decoding is skipped after 1.5 s of silence; audio is still buffered, so nothing is lost
Streaming ASR — Hindi/Hinglish	Zipformer2-CTC, int8, Devanagari characters (`betterflow-hindi-streaming-ctc`, Apache-2.0)	writes English loan words in Devanagari; the cue lexicon covers both scripts
Streaming ASR — English	Zipformer transducer 20M, int8 (k2-fsa)	host RTF 0.05; on-device RTF from the perf overlay
Endpointing	trailing silence 0.8 s, or 12 s maximum utterance	each final segment triggers an immediate engine tick

Design decision from this phase. The proposal named a Hindi FastConformer streaming model. Its maintainers document that it needs a patched sherpa-onnx feature extractor and cannot write Latin script (12 WER points worse on Hinglish). We therefore ship the Zipformer2-CTC model, which runs on the stock

runtime and handles Hinglish. The NPU export of speech models (ExecuTorch Qualcomm/MediaTek backends) is Phase 2.

3.2 Pipeline B — reasoning (Llama 3.2 1B, ExecuTorch, Arm CPU)

Item	Choice
Model	Llama-3.2-1B-Instruct + KAVACH LoRA (r = 16, all projections), merged before export
Runtime	ExecuTorch `LlmModule` (Android AAR), XNNPACK delegate; KleidiAI int4 micro-kernels are enabled by default on Arm
Quantisation (shipped)	8da4w: 4-bit groupwise weights (g = 32), 8-bit dynamic activations, 4-bit embeddings
When it runs	only on escalation to CAUTION/STOP; never in the per-segment loop
Latency tricks	**warm prefix** (constant system prompt pre-filled into the KV cache at start and re-armed in the background after every alert); **early stop** as soon as the JSON object closes; greedy decoding; 8 s hard timeout
Thread	dedicated `kavach-llm` thread at urgent priority, so the scheduler places bursts on big cores; placement is logged from `/proc/self/task/*/stat`
Contract	ONE JSON object: `scam, type, stage, confidence, evidence[≤3 verbatim quotes], say(≤35 words), action`
Validator	enums enforced, confidence clamped, **quotes not found verbatim in the transcript are dropped** (and confidence capped), one re-prompt on invalid output
Authority	may raise CAUTION → STOP ("llm-early-warning") at confidence ≥ 0.8 and stage ≥ isolation; can never lower a rule STOP

3.3 The always-on fast path (both phones)

1. **Cue lexicon** (`assets/cues.json`): 583 phrases in 13 categories (authority, identity misuse, threat, urgency, isolation, credential request, remote access, link, payment, reward, family emergency, safety advice, …), each in English, romanised Hinglish and Devanagari. The phrase rules support prefixes and gaps of 0–4 words. A request is ignored when a nearby word negates it ("never share your OTP" is *safety advice*, not a request).
2. **Hashed linear classifiers** (`cue_model`, `sms_model`, `url_model`). Features: FNV-1a hashed words, word pairs and 3–5 letter fragments, plus cue-category hits and category pairs. The models are 1 MB each, with identical inference in Python and Kotlin. Every score breaks down into per-word contributions, which become the "evidence" words shown to the user.
3. **Kill-chain** (`killchain.py` / `KillChain.kt`): evidence fades with a half-life that depends on the channel (10 min for calls, 30 min for SMS and links, 24 h for apps). The level (none → info → caution → stop) is *sticky* within a session: a scammer cannot calm the guard with small talk. STOP rules: isolation + authority; authority + identity + threat; two different action/payment requests; a hook plus an action with no genuine safety advice heard; cross-channel evidence (a risky call plus a dangerous link, APK or screen share); total score.
4. **Guardian logic**: ticks after every final ASR segment and at least every 3 s. The classifier adds CAUTION or STOP when it stays above 0.8 for two consecutive ticks. It can also **veto** a weak rule-only STOP while it

has been confidently benign (< 0.5); strong rules are never vetoed. Thresholds are tuned on the development sets only (`bench/results/guardian_tuning.json`).

3.4 User's phone modules

- **Message Shield** reads SMS/WhatsApp/Telegram *notifications*, so it needs no READ_SMS permission. It runs the SMS classifier, the cues and Link Check on every URL.
- **Link Check** checks, in order: an offline allow-list of genuine Indian domains, including the RBI-mandated `.bank.in` and `.fin.in` suffixes; then look-alike rules (brand name in an unofficial domain, typosquat by edit distance, homoglyph or punycode, shortener, raw IP, `@` trick, APK download, scam-prone TLD, brand page on a free website builder); then the URL model with a threshold chosen on dev for ≤ 3 % false alarms.
- **Kill-chain hub** fuses messages, links, apps, screen sharing and the Guardian's call level. It raises a full-screen STOP card (spoken, with a one-tap call to 1930) and can optionally SMS a family member.
- **App Inspector v0** (preview) scores the permission profile of each installed or new app: SMS + Accessibility + overlay, sideloaded, named like a bank/KYC/challan app. **Screen-share Guard v0** (preview) raises STOP when a payment or banking app comes to the foreground while a remote-access app is active.
- **Device link**: Bluetooth RFCOMM carrying newline-delimited JSON events when the Guardian is another phone. A **same-phone link** (explicit broadcasts protected by a signature-level permission) is used when Guardian and Shield are installed on one Android phone and the call is on another phone, e.g. an iPhone, on speaker. This is our team's demo setup. Both work offline.
- **Own-voice filter**: the Guardian ignores ASR segments recognised while it is speaking a warning and for 1.2 s afterwards. Otherwise its own warning ("police", "arrest", "1930") could raise its level.

4. Arm-specific engineering

Lever	Status in Phase 1	How it is measured
ExecuTorch + XNNPACK, `.pte` export	export ladder verified on a tiny Llama-3.2 model; Android integration compiled against `LlmModule`	`models/llama/tiny_pipeline_check.py`, `bench/results/llama_pipeline_check.json`
KleidiAI int4 kernels on vs off	build script for two `llama_main` binaries (`EXECUTORCH_XNNPACK_ENABLE_KLEIDI=ON/OFF`)	`bench/build_llama_runner.sh`, `bench/llama_bench.sh` (same model, prompt and phone)
Quantisation ladder	bf16 → 8da4w g128 + 8-bit embeddings → 8da4w g32 + 4-bit embeddings	`models/llama/export_ladder.sh` + `eval_llama.py` (quality) + `llama_bench.sh` (speed, memory)
big.LITTLE placement	`taskset` masks for big, little and all cores × 2 or 4 threads in the bench; the app logs the core each thread last ran on	`bench/llama_bench.sh`, Guardian session CSV
Battery and thermal	Guardian writes a perf row every 5 s: per-core clock, battery %, °C, current, thermal status, PSS, ASR RTF, Llama TTFT and tok/s	`bench/parse_session.py`, `bench/thermal_soak.sh` (30 min)
Prefix caching	warm system prefix in the KV cache via `prefillPrompt`; falls back automatically if a runtime build lacks text prefill	Llama stats in the session log (`usedWarmPrefix`)
Fast path on LITTLE cores	0.44 ms per tick (lexicon + kill-chain + classifier over 120 tokens) in the shipped Kotlin code on a laptop JVM	`bench/results/kotlin_engine_host.json`; on-device numbers from the perf overlay

One finding from this phase: `qmode: int8` fails to export on the GQA projections in ExecuTorch 1.5.1, and `torchao:8da8w` cannot be combined with XNNPACK. The ladder's middle rung is therefore 4-bit with coarse groups. Both failures were reproduced on the tiny model.

5. Data

Need	Source	Use
Call transcripts	**Synthetic generator** (`data/synth/generate_calls.py`): 11 scam scripts and 9 benign scenarios built from 41 utterance pools. Each pool has an English, Hinglish and Devanagari rendering, with ASR-style corruption and realistic timing. Scripts are paraphrased from I4C, RBI and bank advisories.	train 8,000 / dev 600 / test 1,000 calls
Hard negatives	genuine bank fraud alerts that say "never share OTP", delivery agents asking for the delivery OTP, relatives asking for money, people *talking about* scams, genuine police calls, office talk about payments and screen sharing	in every split
Held-out phrasing	**template-disjoint splits**: each pool is partitioned by item into train, dev and test, so dev and test contain phrasings never trained on	model selection on dev; test reported once
Independent checks	**30 hand-written calls** (a different author style) and **20 locked held-out calls**, written before any tuning against them and used only for reporting	`data/dev/`
Real ASR noise	88 English training calls synthesised with Piper TTS, passed through a speaker-phone channel, transcribed by the Guardian's ASR, with stage labels aligned by time	`models/asr/make_asr_noisy_calls.py`
SMS	UCI SMS Spam Collection (CC BY 4.0) + 4,300 synthetic Indian scam and genuine messages (KYC, e-challan APK, electricity, parcel fee, UPI refund, jobs, loans vs real OTP and debit-alert formats)	`sms_model`
URLs	Phishing.Database active phishing domains (MIT, 40k sampled), top-10k popular domains, the genuine Indian domain list, 8k generated Indian look-alikes; split by registrable domain	`url_model`
Llama fine-tuning	4,000 / 400 instruction pairs generated by the same prompt code the phone uses (`models/llama/make_sft_data.py`)	LoRA

No real victim audio or personal data is used or committed. The final test set must be **human-recorded** role-play calls (`demo/recording_protocol.md`).

6. Engineering quality

- **Single source of truth**: lexicon, playbook, domain lists and model weights live in `assets/` and are packaged into both APKs by Gradle.
- **Parity**: `tests/make_parity_vectors.py` writes golden outputs from Python, and `android/core` checks 196 of them. The checks cover normalisation, hashing, lexicon hits, feature vectors, logits of all three models, URL verdicts, message verdicts, kill-chain steps, retrieval, prompts, JSON validation and full Guardian replays. Porting found and fixed two real bugs: Python's brand ordering depended on hash randomisation, and Java and Python IDNA decoders disagree on some punycode.
- **Unit tests**: 22 pytest tests. Writing them found a safety bug: `"scam": "maybe"` was being read as *not a scam*; it now fails validation and triggers the re-prompt.
- **Fail-safe by construction**: if the model is missing, slow or throttled, the rule + classifier path still speaks a warning, which is generated from the vetted playbook.
- **Privacy by construction**: no `INTERNET` permission, audio never stored, transcript kept in memory only, notifications read without SMS permission.

7. Results

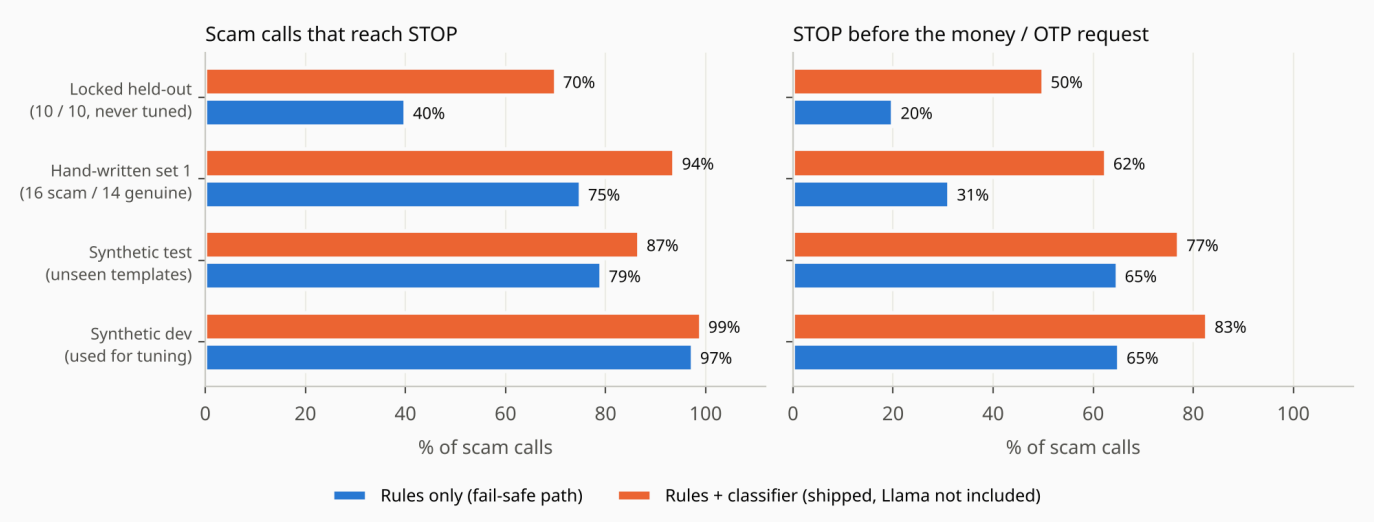
7.1 Text classifiers (held-out test splits)

Model	Task	Test n	Precision	Recall	F1	ROC-AUC
`cue_model`	120-token call window → scam	8,755	0.935	0.873	**0.903**	0.953
`sms_model`	SMS / notification → scam	1,914	0.994	0.897	**0.943**	0.990
`url_model` (rules + model)	URL → phishing	10,196	0.987	0.752	0.853	0.945

Model selection on a *template-disjoint* dev split mattered. With a dev split that shared templates with training, the cue model looked perfect on dev (F1 0.999) but reached only F1 0.817 on test. Adding cue-category features raised test F1 to 0.903.

Link Check flags 98.5 % of generated Indian look-alikes, **0 %** of genuine Indian domains, 3.9 % of popular global domains and 69 % of generic global phishing domains (many are random strings with no brand cue; at a ≤3 % false-alarm budget these stay "unknown").

7.2 Call Guardian (call level, replayed in call time)



Call-level results

Set	Config	STOP recall	≥ CAUTION	STOP before money	Median time to STOP from first scam cue	False STOP	False CAUTION
Synthetic test (503 scam / 497 genuine)	rules only	79.1 %	86.1 %	64.8 %	16.7 s	0 %	0 %
	shipped	**86.7 %**	96.0 %	77.1 %	12.8 s	**0 %**	8.3 %
Hand-written (16 / 14)	rules only	75.0 %	87.5 %	31.3 %	24.7 s	14.3 %	21.4 %
	shipped	**93.8 %**	93.8 %	62.5 %	19.1 s	**0 %**	28.6 %
Locked held-out (10 / 10)	rules only	40 %	70 %	20 %	21.9 s	0 %	10 %
	shipped	**70 %**	80 %	50 %	22.6 s	**0 %**	40 %

The proposal targets (median ≤ 30 s to STOP and ≤ 2 % false STOPs) are met on every set. The ≥ 90 % "caught before payment" target is **not met yet** (50–77 %), and false CAUTIONs on genuine calls are high on the small hand-written sets. Both are the job the Llama layer was designed for, and we will measure it in Phase 2 rather than claim it now. The classifier veto removed both rule-only false STOPs on the hand-written set (people *discussing* a scam; an office call about screen sharing and payments).

7.3 End-to-end audio (host)

The scripted calls are synthesised with Piper TTS, passed through an 8 kHz telephone channel (300–3,400 Hz), room reverb and noise at 30, 15 and 8 dB SNR, then streamed into the Guardian's English ASR and engine:

Call	WER (30 / 15 / 8 dB)	Level reached
Digital arrest	0.38 / 0.49 / 0.61	STOP / STOP / CAUTION
Fake KYC	0.30 / 0.26 / 0.49	STOP / STOP / CAUTION
Genuine bank fraud alert	0.41 / 0.45 / 0.54	none / none / none
Delivery agent asking for OTP	0.27 / 0.32 / 0.59	none / info / none

ASR real-time factor on the host is 0.05–0.07. The small generic English model mishears Indian scam vocabulary (“money wandering” for *laundering*, “arbi” for *RBI*, “o p” for *OTP*). We also trained the cue model on real ASR transcripts of training calls and compared the two models on **identical saved audio**: one of the six scam runs improved (digital arrest at 15 dB, INFO → CAUTION), none got worse, and genuine calls were unchanged (`bench/results/e2e_audio_paired.json`). One fake-KYC run at 8 dB still only reaches INFO. The table above comes from a fresh audio run; Piper TTS is stochastic, so levels vary between runs. The real fix is better ASR (§8).

Negative result, kept out of the product. We tried “acoustic lexicon expansion”: synthesise each cue phrase on its own and add its ASR misrecognitions as aliases. On short isolated phrases the streaming model returned word fragments (“ation”, “ly”), which would cause false alarms, so the aliases were discarded.

7.4 Llama pipeline verification

Check	Result
HF → Meta weight conversion (tiny Llama 3.2-shaped)	max $\lvert\Delta\text{logit}\rvert$ 3.6×10^{-7}
fp32 `.pte` with KV cache and custom SDPA vs HF, step by step	max $\lvert\Delta\rvert$ 4.8×10^{-7}, 100 % next-token agreement
shipped 8da4w `.pte` vs HF	100 % next-token agreement
Real alert prompts	1,361-character constant prefix + 1.7–2.0 k-character suffix (`bench/prompts/`)

7.5 Still to measure on phones (Phase 2 week 1–2, scripts ready)

Llama TTFT, decode tok/s and RAM per quantisation rung, with KleidiAI on and off, on big and LITTLE cores, on a flagship and a budget phone. Also: Hindi ASR WER on our own speaker-phone recordings, battery drain per hour of guarding, the 30-minute thermal soak, the Llama verdict quality and escalation effect on the locked held-out calls, and the human-recorded test set. Reference point: ExecuTorch reports about 50 tok/s decode for Llama 3.2 1B SpinQuant on a OnePlus 12.

8. Limitations and threat model (summary)

- **ASR is the dominant error source.** Speaker-phone Hindi WER is unknown until recorded. Mitigations are a Devanagari lexicon, training on ASR-noisy text, Llama reasoning over noisy text, and adaptation on our own recordings (Phase 2).
- **Synthetic training data**, written by the team. The held-out and hand-written sets reduce but do not remove author bias; the human-recorded test set is the real measure.
- **Adversaries** can avoid trigger words. The classifier, the cue-category combinations and Llama generalise beyond exact phrases, and cross-channel fusion catches the action (link, APK, screen share) even when the talk is clean.
- **Platform policy**: notification access, usage access and `QUERY_ALL_PACKAGES` suit a side-loaded prototype; `docs/threat_model.md` describes a Play-policy-compliant design.
- **The Guardian is a second phone**; the production path is dialer integration.

9. Phase 2 plan

1. Fine-tune the LoRA and export the quantisation ladder. Measure quality (`eval_llama.py`), on-device speed and the **Llama escalation effect** on all call sets.

2. Record 100+ scam and 100+ genuine role-play calls (consented), measure Hindi WER, adapt ASR, and build the untouched human-recorded test set.
3. NPU export of the speech models (ExecuTorch QNN/MediaTek) and a CPU vs NPU energy comparison.
4. Gradient-boosted App Inspector on public malware data; local-VPN Network Watch (DGA and beaconing).
5. Budget-phone port, 30-minute thermal soak, and a consent-based pilot with 20–30 family volunteers.

10. Reproducing everything

```
        pip install numpy scipy scikit-learn pytest           # core + tests
        ./data/public/download.sh                             # public datasets
        python data/synth/generate_calls.py && python data/synth/generate_sms.py && python
data/dev/build_handwritten.py
        python models/classifiers/train_text.py && python models/classifiers/train_url.py
        python bench/eval_calls.py --split test               # + dev / handwritten / heldout
        python -m pytest -q tests && python tests/make_parity_vectors.py
        cd android && ./gradlew :core:test                     # Kotlin parity (196 checks)
        scripts/build_apks.sh                                 # debug APKs (needs Android SDK)
        pip install executorch==1.4.0 transformers && python models/llama/tiny_pipeline_check.py
        pip install sherpa-onnx && python bench/e2e_audio.py --models <asr+tts model dir>
```

Sources: proposal §Sources (MHA/I4C figures, ExecuTorch Llama README, sherpa-onnx, RBI circular RBI/2025-26/28 on `.bank.in`), the model cards in `docs/model_cards.md` and the data card in `docs/data_card.md`.